



COURSE DESCRIPTION FORM: CL-1004: Object Oriented Programming Lab

INSTITUTION FAST School of Computing, National University of Computer and Emerging Sciences, Islamabad Campus

PROGRAM TO BE EVALUATED **BS Computer Science: Spring 2024**

Course Description

Course Code	CL-1004		
Course Title	Object-Oriented Programming Lab		
Credit Hours	1		
Course Instructors	Ms. Bushra Fatima, Mr. Usama Bin Imran, Ms. Shahnoor Haider		
Grading Policy	Absolute Grading		
Policy about missed assessment items in the course	Retake of missed assessment items (other than sessional/ final exam) will not be held. Student who misses an assessment item (other than sessional / final exam) is awarded zero marks in that assessment item i.e. late submission will not be accepted. For missed sessional/ final exam, exam retake/ pre-take application along with necessary evidence are required to be submitted to the department secretary. The examination assessment and retake committee decides the exam retake/ pre-take cases.		
Course Plagiarism Policy	Plagiarism in project or final exam will result in F grade in the course. Plagiarism in a Lab-Task, Quiz, Assignment will result in zero marks in the whole category.		
Policy about late submission of assessment item	Penalties for late submissions of assignment/project: - Up to 15 mins, loss of 20% of the mark awarded. - After 15 mins, assignment/project will not be accepted for marking.		
Prerequisites by Course(s) or Topics	CS-1002, CL-1002		
Assessment Instruments with Weights (Final exam, lab Project, lab tasks, etc.)	Assessment with the weight.		
	Assessment Item	Number	Weight (%)
	Lab Tasks	15	35
	Sessional	1	15
	Final Exam	1	50
Course Coordinator	Ms. Bushra Fatima		
URL (if any)			

Course Catalog Description	The course aims to provide students with the ability to analyze the given requirements for solving problems in different domains and to implement the solutions on a computer system. It mainly emphasizes applying the object-oriented programming (OOP) principles and the need to separate the interface from the implementation. The students will learn the syntax and control structures of the C++ programming language for the implementation.																																	
Laboratory Manual	Tony Gaddis "Starting Out with C++ from Control Structures to Objects" 8 th Edition																																	
Course Goals	<table border="1" style="width: 100%;"> <tr> <th align="left" colspan="3">A. Course Learning Outcomes (CLOs)</th></tr> <tr> <td colspan="3">After completion of the course, the students shall be able to:</td></tr> <tr> <td colspan="3"> 1. Understand the object-oriented programming principles 2. Design a software system of a reasonable size for a real-life problem 3. Apply the OOP principal to implement a project and formally present the work. </td></tr> <tr> <th align="left" colspan="3">B. Program Learning Outcomes (PLOs)</th></tr> <tr> <td colspan="3"> <table border="1" style="width: 100%;"> <tr> <td style="width: 20%;">1. Computing Knowledge:</td><td style="width: 60%;">Apply knowledge of mathematics, natural sciences, computing fundamentals, and a computing specialization to the solution of complex computing problems.</td><td style="width: 20%; text-align: center;">✓</td></tr> <tr> <td>2. Problem Analysis:</td><td>Identify, formulate, research literature, and analyze complex computing problems, reaching substantiated conclusions using first principles of mathematics, natural sciences, and computing sciences.</td><td style="text-align: center;">✓</td></tr> <tr> <td>3. Design/Develop Solutions:</td><td>Design solutions for complex computing problems and design systems, components, and processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.</td><td style="text-align: center;">✓</td></tr> <tr> <td>4. Investigation & Experimentation</td><td>Conduct investigation of complex computing problems using research-based knowledge and research-based methods.</td><td></td></tr> <tr> <td>5. Modern Tool Usage:</td><td>Create, select, and apply appropriate techniques, resources, and modern computing tools, including prediction and modelling for complex computing problems.</td><td style="text-align: center;">✓</td></tr> <tr> <td>6. Society Responsibility:</td><td>Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to context of complex computing problems.</td><td></td></tr> </table> </td></tr> </table>	A. Course Learning Outcomes (CLOs)			After completion of the course, the students shall be able to:			1. Understand the object-oriented programming principles 2. Design a software system of a reasonable size for a real-life problem 3. Apply the OOP principal to implement a project and formally present the work.			B. Program Learning Outcomes (PLOs)			<table border="1" style="width: 100%;"> <tr> <td style="width: 20%;">1. Computing Knowledge:</td><td style="width: 60%;">Apply knowledge of mathematics, natural sciences, computing fundamentals, and a computing specialization to the solution of complex computing problems.</td><td style="width: 20%; text-align: center;">✓</td></tr> <tr> <td>2. Problem Analysis:</td><td>Identify, formulate, research literature, and analyze complex computing problems, reaching substantiated conclusions using first principles of mathematics, natural sciences, and computing sciences.</td><td style="text-align: center;">✓</td></tr> <tr> <td>3. Design/Develop Solutions:</td><td>Design solutions for complex computing problems and design systems, components, and processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.</td><td style="text-align: center;">✓</td></tr> <tr> <td>4. Investigation & Experimentation</td><td>Conduct investigation of complex computing problems using research-based knowledge and research-based methods.</td><td></td></tr> <tr> <td>5. Modern Tool Usage:</td><td>Create, select, and apply appropriate techniques, resources, and modern computing tools, including prediction and modelling for complex computing problems.</td><td style="text-align: center;">✓</td></tr> <tr> <td>6. Society Responsibility:</td><td>Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to context of complex computing problems.</td><td></td></tr> </table>			1. Computing Knowledge:	Apply knowledge of mathematics, natural sciences, computing fundamentals, and a computing specialization to the solution of complex computing problems.	✓	2. Problem Analysis:	Identify, formulate, research literature, and analyze complex computing problems, reaching substantiated conclusions using first principles of mathematics, natural sciences, and computing sciences.	✓	3. Design/Develop Solutions:	Design solutions for complex computing problems and design systems, components, and processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.	✓	4. Investigation & Experimentation	Conduct investigation of complex computing problems using research-based knowledge and research-based methods.		5. Modern Tool Usage:	Create, select, and apply appropriate techniques, resources, and modern computing tools, including prediction and modelling for complex computing problems.	✓	6. Society Responsibility:	Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to context of complex computing problems.	
A. Course Learning Outcomes (CLOs)																																		
After completion of the course, the students shall be able to:																																		
1. Understand the object-oriented programming principles 2. Design a software system of a reasonable size for a real-life problem 3. Apply the OOP principal to implement a project and formally present the work.																																		
B. Program Learning Outcomes (PLOs)																																		
<table border="1" style="width: 100%;"> <tr> <td style="width: 20%;">1. Computing Knowledge:</td><td style="width: 60%;">Apply knowledge of mathematics, natural sciences, computing fundamentals, and a computing specialization to the solution of complex computing problems.</td><td style="width: 20%; text-align: center;">✓</td></tr> <tr> <td>2. Problem Analysis:</td><td>Identify, formulate, research literature, and analyze complex computing problems, reaching substantiated conclusions using first principles of mathematics, natural sciences, and computing sciences.</td><td style="text-align: center;">✓</td></tr> <tr> <td>3. Design/Develop Solutions:</td><td>Design solutions for complex computing problems and design systems, components, and processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.</td><td style="text-align: center;">✓</td></tr> <tr> <td>4. Investigation & Experimentation</td><td>Conduct investigation of complex computing problems using research-based knowledge and research-based methods.</td><td></td></tr> <tr> <td>5. Modern Tool Usage:</td><td>Create, select, and apply appropriate techniques, resources, and modern computing tools, including prediction and modelling for complex computing problems.</td><td style="text-align: center;">✓</td></tr> <tr> <td>6. Society Responsibility:</td><td>Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to context of complex computing problems.</td><td></td></tr> </table>			1. Computing Knowledge:	Apply knowledge of mathematics, natural sciences, computing fundamentals, and a computing specialization to the solution of complex computing problems.	✓	2. Problem Analysis:	Identify, formulate, research literature, and analyze complex computing problems, reaching substantiated conclusions using first principles of mathematics, natural sciences, and computing sciences.	✓	3. Design/Develop Solutions:	Design solutions for complex computing problems and design systems, components, and processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.	✓	4. Investigation & Experimentation	Conduct investigation of complex computing problems using research-based knowledge and research-based methods.		5. Modern Tool Usage:	Create, select, and apply appropriate techniques, resources, and modern computing tools, including prediction and modelling for complex computing problems.	✓	6. Society Responsibility:	Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to context of complex computing problems.															
1. Computing Knowledge:	Apply knowledge of mathematics, natural sciences, computing fundamentals, and a computing specialization to the solution of complex computing problems.	✓																																
2. Problem Analysis:	Identify, formulate, research literature, and analyze complex computing problems, reaching substantiated conclusions using first principles of mathematics, natural sciences, and computing sciences.	✓																																
3. Design/Develop Solutions:	Design solutions for complex computing problems and design systems, components, and processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.	✓																																
4. Investigation & Experimentation	Conduct investigation of complex computing problems using research-based knowledge and research-based methods.																																	
5. Modern Tool Usage:	Create, select, and apply appropriate techniques, resources, and modern computing tools, including prediction and modelling for complex computing problems.	✓																																
6. Society Responsibility:	Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues relevant to context of complex computing problems.																																	

	7. Environment and Sustainability:	Understand and evaluate sustainability and impact of professional computing work in the solution of complex computing problems.																																																																																							
	8. Ethics:	Apply ethical principles and commit to professional ethics and responsibilities and norms of computing practice.																																																																																							
	9. Individual and Teamwork:	Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.																																																																																							
	10. Communication:	Communicate effectively on complex computing activities with the computing community and with society at large.																																																																																							
	11. Project Management and Finance:	Demonstrate knowledge and understanding of management principles and economic decision making and apply these to one's own work as a member or a team.																																																																																							
	12. Lifelong Learning:	Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological changes.																																																																																							
<table border="1"> <tr> <th colspan="12">C. Mapping of CLOs to PLOs (CLO: Course Learning Outcome, PLOs: Program Learning Outcomes)</th> </tr> <tr> <th colspan="2" rowspan="2"></th> <th colspan="10">PLOs</th> </tr> <tr> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> </tr> <tr> <td rowspan="4">CLOs</td> <td>1</td> <td>✓</td> <td>✓</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> </tr> <tr> <td>2</td> <td></td> <td>✓</td> <td>✓</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> </tr> <tr> <td>3</td> <td></td> <td></td> <td>✓</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td>✓</td> <td></td> <td></td> </tr> <tr> <td>4</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> </tr> </table>				C. Mapping of CLOs to PLOs (CLO: Course Learning Outcome, PLOs: Program Learning Outcomes)														PLOs										1	2	3	4	5	6	7	8	9	10	11	12	CLOs	1	✓	✓											2		✓	✓									3			✓						✓			4											
C. Mapping of CLOs to PLOs (CLO: Course Learning Outcome, PLOs: Program Learning Outcomes)																																																																																									
		PLOs																																																																																							
		1	2	3	4	5	6	7	8	9	10	11	12																																																																												
CLOs	1	✓	✓																																																																																						
	2		✓	✓																																																																																					
	3			✓						✓																																																																															
	4																																																																																								
Topics covered in the course (assume 15-week instruction and 3 contact hours per week)																																																																																									

	1. Topics to be covered:			
	List of Topics	No. of Weeks	Contact Hours	CLO(s)
	Tools setup (gtest, Visual Studio for C++)	1	3	
	Pointers	2	3	1,2,3
	Recursion	2	3	1,2,3
	Classes-I (definition, constr, dest, overloaded contrs)	1	3	1,2,3
	Classes-II (this pointer, initializer list, inline functions)	1	3	1,2,3
	Classes-III (constant & nonconstant members, static data members)	1	3	1,2,3
	Operator Overloading-I (function overloading and basic op. overloading)	1	3	1,2,3
	Operator Overloading-II (operator overloading as members)	1	3	1,2,3
	Operator Overloading-III (non-member and stream ops)	1	3	1,2,3
	Composition, Aggregation	1	3	1,2,3
	Inheritance	1	3	1,2,3
	Polymorphism	1	3	1,2,3
	Templates	1	3	1,2,3
	Total	15	45	
Practical/ Programming Work/ Tools	Visual Studio C++ gtest			
Lab Time Spent (in percentage)	Theory	Problem Analysis	Solution Design	Social and Ethical Issues
	5	15	75	5